

## 1. Git Merge

### What is Git Merge?

git merge is used to integrate changes from different branches into a single branch. It combines the content of one branch into another, often the main branch (main or master).

### How to Use Git Merge

1. **Check the Current Branch** Before merging, make sure you are on the branch that you want to merge into:

```
git branch
```

2. **Switch to the Target Branch** Switch to the branch you want to merge into (e.g., main):

```
git checkout main
```

3. **Merge the Branch** Merge the changes from the other branch (e.g., feature-branch) into the current branch:

```
git merge feature-branch
```

4. **Resolve Conflicts (If Any)** If there are conflicts, Git will prompt you to resolve them. Open the conflicting files, make the necessary edits, and then:

```
git add <file>
```

```
git commit
```

### Example Workflow

- Create a new branch:

```
git checkout -b feature-branch
```

- Make changes, add, and commit:

```
git add .
```

```
git commit -m "Add new feature"
```

- Switch back to the main branch and merge:

```
git checkout main
```

```
git merge feature-branch
```

## 2. Git Pull

### What is Git Pull?

git pull is used to fetch and merge changes from a remote repository into your local repository. It is a combination of **git fetch** and **git merge**.

### How to Use Git Pull

1. **Ensure You Are on the Correct Branch** Make sure you are on the branch you want to update:  
`git checkout main`
2. **Pull the Latest Changes** Pull the latest changes from the remote repository:
3. `git pull origin main`

### Example Workflow

- Suppose you have a remote repository with changes made by other team members:  
`git pull origin main`

## 2. Git Fetch

git fetch downloads the **latest commits** from a remote repository (like GitHub), **but does not change** your working directory or local branch.

1. **Fetches updates from the default remote (origin)**  
`git fetch`
2. **Fetches updates from the default specific branch**  
`git fetch origin <<branch_name>>`

## 3. Git PULL equivalent ( GIT Fetch + Git Merge )

1. **Fetches updates from the default specific branch**  
`git fetch origin <<branch_name>>`
2. **Merge Remote changes to Local repo**  
`git merge origin/<<branch_name>>`

## 4. Git Stash

### What is Git Stash?

git stash temporarily saves your changes without committing them. This is useful if you want to switch branches or pull changes without committing your current work.

### How to Use Git Stash

1. **Stash Your Changes** Save your uncommitted changes:

```
git stash
```

2. **List Stashes** View a list of all saved stashes:

```
git stash list
```

3. **Apply a Stash** Apply the latest stash or a specific stash ( show the changes back again )

```
git stash apply
```

### Example Workflow

- Make some changes, but don't want to commit:

```
git stash
```

- Later, apply the stashed changes:

```
git stash apply
```

### Follow the steps to learn more:

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (master) #  
check how many branches
```

```
$ git branch
```

```
feature1
```

```
* master
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (master) #  
checkout feature1 branch
```

```
$ git checkout feature1
```

```
Switched to branch 'feature1'
```

hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (feature1)  
# Check all the files in feature1 branch

\$ ls

mergeFile.text newFile.text pullrequest.text testFile.text

hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (feature1) #  
Add new files in feature1 branch

\$ touch stashcheck.text

hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (feature1)

\$ git status

On branch feature1

Untracked files:

(use "git add <file>..." to include in what will be committed)

stashcheck.text

nothing added to commit but untracked files present (use "git add" to track)

hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (feature1)  
# Add new files to git but DON'T COMMIT

\$ git add .

hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (feature1)

\$ git status

On branch feature1

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

new file: stashcheck.text

hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (feature1)

# Checkout to master branch

\$ git checkout master

Switched to branch 'master'

A       stashcheck.text

Your branch is up to date with 'origin/master'.

\$ ls

mergeFile.text newFile.text pullrequest.text stashcheck.text testFile.text

hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (master) #

**Check the files in master branch and surprisingly see the stash file . ideally this file should not be available in master branch because it was created in feature1 branch.**

**This is because we didn't save the changes to Local Repo hence its overflowing in master branch as well. So solve this problem we will use GIT STASH**

hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (master)

\$ git checkout feature1

Switched to branch 'feature1'

A       stashcheck.text

hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (feature1)

\$ git stash

Saved working directory and index state WIP on feature1: e299a15 new file added in feature1

hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (feature1)

\$ git checkout master

Switched to branch 'master'

Your branch is up to date with 'origin/master'.

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (master) #  
Now the stash file is not appearing
```

```
$ ls
```

```
mergeFile.text  newFile.text  pullrequest.text  testFile.text
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (master) #  
Now move to feature1
```

```
$ git checkout feature1
```

```
Switched to branch 'feature1'
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (feature1)  
# Now stash apply to bring back that file for further update in the file
```

```
$ git stash apply
```

```
On branch feature1
```

```
Changes to be committed:
```

```
(use "git restore --staged <file>..." to unstage)
```

```
    new file:   stashcheck.text
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (feature1)
```

```
$ ls
```

```
mergeFile.text  newFile.text  pullrequest.text  stashcheck.text  testFile.text
```

## 5. Git Clone

### What is Git Clone?

git clone is used to create a copy of a remote repository on your local machine.

### How to Use Git Clone

1. Clone a repository (**default branch ( main/master )** ) of a remote URL:

```
git clone https://github.com/user/repository.git
```

1. Clone a repository (**Specific** ) of a remote URL:

```
git clone -b <<branch_name>> https://github.com/etlqalabs/Sep2025Batch.git
```

e.g.

```
git clone -b branch1 https://github.com/etlqalabs/Sep2025Batch.git
```

to get the URL click on **Code** on Remote Repository

etlqalabs / gitdemoAugBatch

<> Code Issues Pull requests 1 Actions Projects Wiki Security

 **gitdemoAugBatch** Public

**Your master branch isn't protected**  
Protect this branch from force pushing or deletion, or require status checks before merging. [View documentation.](#)

master 3 Branches 0 Tags Go to file

 etlqalabs Merge pull request #5 from etlqalabs/dev1

|                                                                                                      |                              |
|------------------------------------------------------------------------------------------------------|------------------------------|
|  .idea             | push all files to repository |
|  mergeFile.text   | change                       |
|  pullrequest.text | new change                   |
|  testFile.text    | dev1 changes                 |

Local

 Clone

HTTPS SSH GitHub CLI

<https://github.com/etlqalabs>

Clone using the web URL.

 Open with GitHub Desktop

2. **Navigate to the Cloned Directory** After cloning, navigate to the repository directory:

```
cd repository
```

### Example Workflow

- Clone a repository to start working:

```
git clone https://github.com/user/repository.git
```

```
cd repository
```

### You can follow these steps for cloning the Repository

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITDemo (dev1)
```

```
$ cd ..
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects
```

```
$ mkdir GITCloneRepo
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects
```

```
$ ls -lrt
```

```
total 84
```

```
drwxr-xr-x 1 hetur 197609 0 Jan 31  2024  myProj/
drwxr-xr-x 1 hetur 197609 0 Jun 13 07:23  pythonProject/
drwxr-xr-x 1 hetur 197609 0 Aug  8 19:26  ETLAutomation/
drwxr-xr-x 1 hetur 197609 0 Aug  9 07:28  AutomationBatch/
drwxr-xr-x 1 hetur 197609 0 Aug 19 16:13  etlqalabs/
drwxr-xr-x 1 hetur 197609 0 Aug 19 16:20  ETLAutomationE2ESetup/
drwxr-xr-x 1 hetur 197609 0 Aug 21 09:14  AutomationBacth1/
drwxr-xr-x 1 hetur 197609 0 Aug 21 19:14  ETLTest/
drwxr-xr-x 1 hetur 197609 0 Aug 22 18:22  ETLTesting_Demo_Thursday_Test/
drwxr-xr-x 1 hetur 197609 0 Aug 25 10:26  DemoThursday/
drwxr-xr-x 1 hetur 197609 0 Aug 25 11:53  'Demo Sunday'/
drwxr-xr-x 1 hetur 197609 0 Aug 26 07:13  ETLTestAutomationFramework/
drwxr-xr-x 1 hetur 197609 0 Aug 30 08:48  Demo_Thursday/
drwxr-xr-x 1 hetur 197609 0 Sep  1 19:52  MiniProject/
drwxr-xr-x 1 hetur 197609 0 Sep  1 22:36  gitClonePractice/
drwxr-xr-x 1 hetur 197609 0 Sep  1 22:36  gitdemoAugBatch/
drwxr-xr-x 1 hetur 197609 0 Sep  2 08:33  GITDemo/
drwxr-xr-x 1 hetur 197609 0 Sep  2 08:58  GITCloneRepo/
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects
```

```
$ cd GITCloneRepo/
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo
```

```
$ git clone https://github.com/etlqalabs/gitdemoAugBatch.git
```

```
Cloning into 'gitdemoAugBatch'...
```

```
remote: Enumerating objects: 80, done.
```

```
remote: Counting objects: 100% (80/80), done.
```

```
remote: Compressing objects: 100% (48/48), done.
```

```
remote: Total 80 (delta 23), reused 67 (delta 18), pack-reused 0 (from 0)
```

```
Receiving objects: 100% (80/80), 12.96 KiB | 1.30 MiB/s, done.
```

```
Resolving deltas: 100% (23/23), done.
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo
```

```
$ ls
```

```
gitdemoAugBatch/
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo
```

```
$ cd gitdemoAugBatch/
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (master)
```

```
$ git branch
```

```
* master
```

```
hetur@DESKTOP-QPJPMJQ MINGW64 ~/PycharmProjects/GITCloneRepo/gitdemoAugBatch (master)
```

```
$ ls
```

```
mergeFile.text  pullrequest.text  testFile.text
```

## Assignments:

### Assignment 3:

1. Repeat the similar exercise on remote repository using GITHUB and :
  - a) Create a merge conflict
  - b) Resolve the merge conflict
  - c) Complete the merge from OOP to master
  
2. Explore the GIT Rebase command as well. Though overall , it has the same purpose like merge, Can you explore the difference how does it internally work and in which scenario we should go for rebase vs merge?